

MACHINE LEARNING
LAPORAN ANALISIS TEKNIS DAN PENJELASAN KODE PROGRAM:
PREPROCESSING, SCALING, DAN REDUKSI DIMENSI DALAM MACHINE
LEARNING



Disusun Oleh:

Nurmelizah – G1A024013

Dosen Pengampu:

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU

2026

A. PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan model pembelajaran mesin (*machine learning*), *preprocessing* (prapemrosesan data) merupakan tahapan kritis yang menentukan kualitas akhir model prediksi. Sebagian besar algoritma berbasis jarak atau optimasi gradien seperti *Support Vector Machine* (SVM), *k-Nearest Neighbors* (k-NN), dan *Principal Component Analysis* (PCA) sangat sensitif terhadap rentang skala antarfitur. Jika suatu fitur memiliki rentang nilai yang jauh lebih besar dibandingkan fitur lainnya, algoritma dapat menganggap fitur tersebut lebih dominan, meskipun secara fungsional nilainya seimbang.

1.2 Tujuan Laporan

Laporan ini disusun secara formal untuk menjelaskan secara rinci maksud, fungsi, serta implikasi teknis dari baris-baris kode yang dijalankan pada *file Jupyter Notebook/Google Colab* yang dilampirkan. Analisis mencakup aspek pemrosesan *dataset Breast Cancer Wisconsin*, eksperimen *scaling* data, efek *data leakage* jika skala diterapkan secara salah, serta visualisasi konsep reduksi dimensi menggunakan PCA.

B. LINGKUP DAN DEPENDENSI SISTEM

Program ini ditulis berbasis bahasa pemrograman Python 3 dengan pustaka utama sebagai berikut:

Nama Pustaka	Fungsi Utama
scikit-learn	Pustaka pembelajaran mesin untuk klasifikasi, pemisahan data (<i>train-test split</i>), transformasi (<i>MinMaxScaler</i> , <i>StandardScaler</i>), dan algoritma klasifikasi (<i>SVC</i>).
mglearn	Pustaka pembantu (<i>helper library</i>) untuk visualisasi konsep <i>Machine Learning</i> secara intuitif.
matplotlib.pyplot	Pustaka grafik dasar untuk visualisasi plot numerik.
numpy & pandas	Pengolahan array multi-dimensi dan struktur data matriks.

C. ANALISIS MENDALAM KODE DAN IMPLEMENTASI

3.1 Bagian 1: Pengaturan Lingkungan & Pustaka Visualisasi (mglearn)

Baris Kode:

```
!pip install mglearn  
import mglearn  
import matplotlib.pyplot as plt
```

Analisis Kode:

1. `!pip install mglearn`: Perintah *shell* untuk mengunduh dan menginstal pustaka mglearn. Pustaka ini dikembangkan oleh Andreas Müller untuk mempermudah visualisasi algoritma *Machine Learning*.
2. `import mglearn & import matplotlib.pyplot as plt`: Memuat modul utama ke dalam memori kerja Python untuk keperluan grafik.
3. `mglearn.plots.plot_scaling()`: Memanggil fungsi visualisasi bawaan yang menampilkan perbandingan 4 metode transformasi data numerik:

3.2 Bagian 2: Pemuatan Dataset dan Pembagian Data (*Train-Test Split*)

Baris Kode:

```
from sklearn.datasets import load_breast_cancer  
from sklearn.model_selection import train_test_split  
cancer = load_breast_cancer()  
X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.target, random_state=1)  
print(X_train.shape)  
print(X_test.shape)
```

Analisis Kode:

- `load_breast_cancer()`: Memuat *dataset* klasifikasi medis kanker payudara yang terdiri dari 569 sampel dan 30 fitur numerik (fitur fisik sel tumor seperti *radius*, *texture*, *perimeter*, dll.).
- `train_test_split(...)`: Membagi data menjadi Data Latih (*Training Set*) dan Data Uji (*Test Set*) secara acak dengan *seed* acak `random_state=1` (rasio bawaan 75% train, 25% test).
- `print(...)` Output:
- `X_train.shape`: (426, 30), 426 sampel latih dengan 30 fitur.
- `X_test.shape`: (143, 30), 143 sampel uji dengan 30 fitur.

3.3 Bagian 3: Penerapan *MinMaxScaler* pada Data Latih dan Data Uji

Baris Kode:

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
scaler.fit(X_train)
# Transformasi Data Latih
X_train_scaled = scaler.transform(X_train)
# Transformasi Data Uji
X_test_scaled = scaler.transform(X_test)
```

Analisis Kode:

1. `scaler.fit(X_train)`: Menghitung parameter statistik minimum dan maksimum untuk setiap fitur hanya dari data latih (`X_train`).
2. `scaler.transform(X_train)`: Menerapkan rumus *MinMaxScaler* ke data latih. Hasilnya, seluruh nilai fitur pada `X_train_scaled` terikat sempurna pada batas minimum 0.0 dan maksimum 1.0.
3. `scaler.transform(X_test)`: Menerapkan skala menggunakan statistik $X_{\min}$ dan $X_{\max}$ yang diambil dari data latih, bukan dari data uji.
4. Analisis Nilai Ekstrem Data Uji:
Setelah scaling, nilai min/max pada data uji (`X_test_scaled`) berada di luar kisaran $[0,1]$ (misalnya $\min = -0.006$ atau $\max = 1.630$). *Hal ini adalah perilaku yang BENAR secara metodologi ilmiah*, karena data uji mencerminkan data masa depan yang belum diketahui distribusinya.

3.4 Bagian 4: Analisis Dampak Kesalahan Skala (*Proper vs Improper Scaling*)

Baris Kode & Visualisasi (Cell 15):

Pada bagian ini dibuat data buatan (*synthetic data*) menggunakan `make_blobs` untuk mendemonstrasikan bahaya jika data latih dan data uji di-scale secara terpisah:

```
# Mensekalakan data uji secara terpisah (SALAH / BAD PRACTICE)
test_scaler = MinMaxScaler()
test_scaler.fit(X_test)
X_test_scaled_badly = test_scaler.transform(X_test)
```

Dampak Pemrosesan Terpisah (Improper Scaling):

- Persebaran Data Terdistorsi: Jika data uji di-fit secara independen (`test_scaler.fit(X_test)`), posisi relatif antara titik data latih dan titik data uji berubah (terjadi rotasi/skalasi lokal yang salah).
- Data Leakage & Bias: Dalam skenario riil, memfit skala pada data uji melanggar aturan utama *machine learning*: model tidak boleh mendapatkan informasi statistik dari data uji sebelum prediksi dilakukan.

3.5 Bagian 5: Efisiensi Komputasi Menggunakan `fit_transform`

Baris Kode:

```
from sklearn.preprocessing import StandardScaler  
  
scaler = StandardScaler()  
  
# Cara manual 2 langkah:  
X_scaled = scaler.fit(X).transform(X)  
  
# Cara efisien (shortcut):  
X_scaled_d = scaler.fit_transform(X)
```

Analisis Kode:

Pustaka scikit-learn menyediakan *method* gabungan `fit_transform()`. Fungsi ini menghitung parameter statistik (rata-rata dan standar deviasi) sekaligus mentransformasikannya dalam satu langkah komputasi yang lebih optimal.

3.6 Bagian 6: Evaluasi Dampak *Preprocessing* Terhadap Performa Model *Supervised Learning* (SVC)

Pada bagian ini, dilakukan pengujian klasifikasi menggunakan algoritma *Support Vector Classifier* (SVC) dengan parameter kriteria kesalahan $C=100$.

Skenario 1: Tanpa Scaling (Data Asli)

```
svm = SVC(C=100)  
  
svm.fit(X_train, y_train)  
  
print("Test set accuracy: {:.2f}".format(svm.score(X_test, y_test)))  
  
Hasil Akurasi: 0.94 (94.0%)
```

Skenario 2: Menggunakan *MinMaxScaler*

```
scaler = MinMaxScaler()
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
svm.fit(X_train_scaled, y_train)
print("Scaled test set accuracy: {:.2f}".format(svm.score(X_test_scaled, y_test)))
Hasil Akurasi: 0.97 (97.0%)
```

Skenario 3: Menggunakan *StandardScaler*

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
svm.fit(X_train_scaled, y_train)
print("SVM test accuracy: {:.2f}".format(svm.score(X_test_scaled, y_test)))
Hasil Akurasi: 0.96 (96.0%)
```

3.7 Bagian 7: Visualisasi dan Konsep *Principal Component Analysis (PCA)*

Baris Kode:

```
mglearn.plots.plot_pca_illustration()
```

Analisis Tahapan Utama PCA:

1. Penemuan Arah Varians Maksimum (*First Principal Component - PC1*): Mencari vektor garis lurus di mana persebaran titik data memiliki rentang varians paling lebar.
2. Penemuan Komponen Kedua (*PC2*): Memilih sumbu tegak lurus (*orthogonal*) terhadap PC1 yang menangkap varians tersisa paling besar.
3. Rotasi Koordinat: Memutar titik-titik data sehingga sumbu PC1 sejajar dengan sumbu horizontal (X) dan PC2 sejajar dengan sumbu vertikal (Y).
4. Reduksi Dimensi (*Dimensionality Reduction*): Memproyeksikan data 2D ke dalam 1D dengan menghapus koordinat PC2 dan hanya mempertahankan PC1.

D. RINGKASAN HASIL EVALUASI MODEL

Tabel berikut merangkum hasil performa klasifikasi *Support Vector Machine* (SVC) berdasarkan teknik prapemrosesan data yang diterapkan:

Skenario Preprocessing	Pustaka / Scaler	Akurasi pada Data Uji (Test Accuracy)	Peningkatan dibanding Baseline
Tanpa Scaling (Raw Data)	Tidak Ada	94.41% (0.94)	Baseline
Penskalaan Min-Max	MinMaxScaler	97.20% (0.97)	+2.79%
Penskalaan Standar	StandardScaler	95.80% (0.96)	+1.39%

E. KESIMPULAN DAN REKOMENDASI

1. Pentingnya Feature Scaling:

Algoritma berbasis jarak seperti *Support Vector Machine* (SVM) sangat bergantung pada *feature scaling*. Tanpa scaling, fitur berukuran numerik besar (seperti *Area* atau *Perimeter*) mendominasi kalkulasi *hyperplane*, sehingga menekan kontribusi fitur bernilai kecil (seperti *Smoothness*).

2. Kesesuaian Metodologi Scaling:

Proses `.fit()` harus selalu dieksekusi hanya pada data latih (`X_train`), kemudian fungsi `.transform()` dipanggil untuk data latih dan data uji (`X_test`). Hal ini bertujuan mencegah *data leakage*.

3. Reduksi Dimensi (PCA):

Principal Component Analysis (PCA) terbukti efektif dalam menyederhanakan variabel berkordinat tinggi dengan tetap mempertahankan informasi varians maksimum.